PHASE

16

Portability: an Architecture Layer

Find every place where x86 leaked into the generic kernel, hide the CPU and the board behind one interface, and prove that x86 still behaves exactly as before

Depends on Phase 15 complete: Kagan 1.0 tagged, `make test` and the fuzzer pass

Delivers The same Kagan on x86, with all x86 code behind `include/arch.h`: a written leak audit in `docs/porting.md`, one trap frame with accessors, `struct bootinfo` instead of Multiboot, the MMU behind `arch_mmu_*`, consoles and input that any board can provide, `make ARCH=`, a 64-bit-clean generic core that compiles for `riscv32` and `aarch64`, a devicetree parser with host tests, and two more cross compilers ready for Phases 17 and 18.

1 Scope and motivation

Kagan 1.0 is x86 from the first instruction to the last trap frame. Some of that is honest: `isr.s`, `gdt.c` and `paging.c` live in `kernel/arch/i386` because they can't be anything but x86. Much of it leaked, though. The system call dispatcher reads `r->eax`, the page-fault handler reads CR2, `fork` writes `tf->eax = 0`, the frame allocator walks the Multiboot memory map, `irq_restore` tests bit `0x200` of EFLAGS, and `kmain` ends a test run by writing to port `0xF4`. Each of these lines works; it only shows up as a leak on a second machine.

Stage V brings that second machine. Phase 17 ports Kagan to RISC-V (RV32 on QEMU's `virt` board, and optionally on the Timur-RV32IMC soft core), and Phase 18 to 64-bit ARM (AArch64 on QEMU's `virt` board, and optionally a Raspberry Pi 4). This phase writes no RISC-V or ARM code. It is a refactoring: it prepares the seams, and at its end Kagan on x86 does exactly what version 1.0 did, with the same tests passing and benchmarks within a few per cent.

Porting sorts the kernel into three kinds of code. The **architecture** is the CPU: instruction set, privilege levels, traps, MMU, atomics. The **board**, or platform, is everything around it: the devices, the firmware, and how the kernel learns what memory and devices exist. Everything else is **generic**. MOS §12.3.1 draws the same split as layers: hide the hardware first, then interrupts, context switching and the MMU, and above that the code is mostly machine independent. The heart of this phase is the **leak audit** of Task 16.2, a written list of every place where x86 shows through; every later task fixes entries on it. Stage V ends with Kagan 2.0 on three architectures.

2 Background

2.1 Architecture, board and generic code

Figure 1 shows where Stage V ends up. Generic code calls the functions of `include/arch.h` and a few inline helpers from per-architecture headers, and three implementations sit below that interface, each on its board. On a PC the architecture and the board are entangled (port I/O is an x86 instruction, the PIC is part of every PC), so Kagan keeps both in `kernel/arch/i386`. The ports separate them: their devices are found at boot, so their drivers can live in `kernel/drivers` and serve any board that has the device.

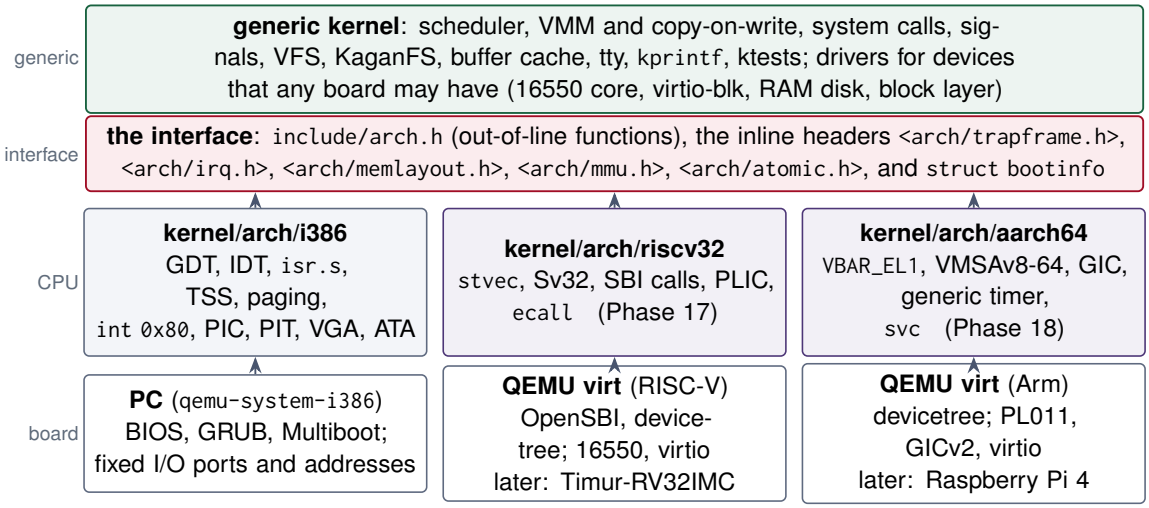


Figure 1: The layers after Stage V. This phase builds the red interface and moves x86 below it; the violet backends come in Phases 17 and 18.

2.2 Three machines, side by side

Table 1 is a map of what the interface hides; Phases 17 and 18 explain every RISC-V and ARM cell. What stays the same matters as much: all three are little-endian, use 4 KiB pages, have one unprivileged mode for programs, and enter the kernel through one trap mechanism for system calls, faults and interrupts.

2.3 One interface, three implementations

There are two ways to keep one source tree for several machines. MOS §12.3.9 (*Hiding the Hardware*) shows **conditional compilation**: `config.h` defines `CPU` and `WORD_LENGTH`, and `#if` blocks select the code for each machine. Its lesson is orthogonality: test CPU-dependent things on the CPU macro and word-length-dependent things on the word-length macro, so that a new CPU with a known word length needs no change to the word-length code. A kernel full of `#if (CPU == ...)` becomes hard to read, though, and every port touches every file. Linux and MINIX 3 use **separate files behind one header** instead: a header declares what every architecture provides, each architecture implements it in its own directory, and the build picks the directory. Kagan does the same, and keeps `#if` only for orthogonal features in Tanenbaum's sense: `ARCH_HAS_ATOMICS` says whether atomic instructions exist, not which CPU this is.

The architecture interface

A fixed set of types, constants and functions that generic code may use and every architecture provides, in two parts. **Inline headers** in `kernel/arch/$(ARCH)/include/arch/` hold what hot paths use or what needs the architecture's types: the trap frame and its accessors, interrupt control, the memory layout, the PTE type. Being `static inline`, they cost nothing at run time. **Out-of-line functions**, declared once in `include/arch.h`, hold the rest: boot, the interrupt controller, the timer, context switching, page tables. A call costs a few nanoseconds, nothing next to what these functions do. Task 16.12 checks both claims.

2.4 The devicetree

A PC kernel simply knows where things are: COM1 at port `0x3F8`, the disk at `0x1F0`, the PIC at `0x20`, and GRUB reports the memory. Other boards have no such conventions, so their firmware passes a **devicetree**: the machine as a tree of **nodes** with named **properties** (SPEC Devicetree Specification, chapter 2). Here is part of the tree that QEMU 10.2 generates for its RISC-V `virt` board:

Mechanism	i386 (Kagan 1.0)	riscv32 (Phase 17)	aarch64 (Phase 18)
Firmware, entry	BIOS and GRUB; Multiboot hands over in protected mode, paging off	OpenSBI runs in M-mode and jumps to the kernel in S-mode, with the hart ID in `a0` and the devicetree's address in `a1`	QEMU's loader (or board firmware) starts the kernel at EL1, or at EL2 when virtualisation is enabled
Privilege levels	ring 0 and ring 3	S-mode and U-mode, with M-mode (firmware) below	EL1 and EL0, with EL2 and EL3 above
Trap entry, fault address	IDT, one gate per vector; CR2	`stvec`, one entry point (direct mode), cause in `scause`; `stval`	`VBAR_EL1`, 16 entries of 128 bytes, cause in `ESR_EL1`; `FAR_EL1`
Interrupt controller	two 8259 PICs	PLIC (claim and complete)	GIC (acknowledge and end of interrupt)
Timer	8254 PIT	SBI timer call (TIME extension) and the `time` counter	Arm generic timer, frequency in `CNTFRQ_EL0`
Page tables	two levels, 32-bit entries, root in CR3	Sv32: two levels, 32-bit entries, root in `satp`	VMSAv8-64, 4 KiB granule, 39-bit addresses: three levels, roots in `TTBR0_EL1` and `TTBR1_EL1`
TLB maintenance	`invlpg`, or reload CR3	`sfence.vma`	`tlbi` between `dsb` barriers, then `isb`
System call	`int 0x80`, number in EAX	`ecall`, number in `a7`	`svc #0`, number in `x8`
Atomics	`xchg`, `lock` prefix	A extension: `amoswap.w`, or LR/SC	`ldaxr/stlxr` pairs (LSE atomics only from Armv8.1; the Cortex-A72 is Armv8.0)
Devices, discovery	port I/O and MMIO; Multiboot and fixed PC addresses	MMIO only; flattened devicetree	MMIO only; flattened devicetree
Word size	32-bit, ILP32	32-bit, ILP32	64-bit, LP64

Table 1: The mechanisms behind the architecture interface, on the three targets of Stage V.

QEMU riscv32 virt devicetree (excerpt in dtc's order; trimmed, comments added)

```
/ {
    #address-cells = <0x02>;
    #size-cells = <0x02>;
    model = "riscv-virtio,qemu";
    memory@80000000 {
        device_type = "memory";
        reg = <0x00 0x80000000 0x00 0x8000000>;   // base 0x80000000, size 128 MiB
    };
    cpus { ... timebase-frequency = <0x989680>; ... };   // the time counter runs at 10 MHz
    chosen { stdout-path = "/soc/serial@10000000"; ... };
    soc {
        #address-cells = <0x02>;
        #size-cells = <0x02>;
        serial@10000000 {
            interrupts = <0x0a>;                  // source 10 of ...
            interrupt-parent = <0x03>;            // ... the node whose phandle is 3
            reg = <0x00 0x10000000 0x00 0x100>;
            compatible = "ns16550a";
        };
        plic@c000000 { phandle = <0x03>; compatible = "sifive,plic-1.0.0", "riscv,plic0"; ... };
    };
};
```

`compatible` names the device, most specific first; a driver looks for a string it knows. `reg` lists (address, size) pairs made of 32-bit cells, and the *parent's* `#address-cells` and `#size-cells` say how many cells each takes: two, 64 bits, even on this 32-bit board. `interrupts` is interpreted by the controller that `interrupt-parent` names through its `phandle`, a numeric node ID. A few nodes have fixed meanings

(chapter 3): `/memory` lists RAM, `/reserved-memory` RAM the kernel must not touch, and `/chosen` the command line (`bootargs`) and the console (`stdout-path`).

The firmware passes the tree as a **flattened devicetree** (FDT, or DTB), the binary format of chapter 5 (Figure 2). A 40-byte header with the magic number `0xd00dfeed` gives the offsets of three blocks. The **structure block** is a stream of 32-bit tokens: `FDT_BEGIN_NODE` (1) and the node's name; `FDT_PROP` (3), the value's length, the offset of the property's name in the **strings block**, and the value; `FDT_END_NODE` (2); `FDT_NOP` (4); and `FDT_END` (9), all padded to four bytes. The **memory reservation block** lists 64-bit (address, size) pairs to keep away from, ending with two zeros. Every number is **big-endian**, and all three Kagan targets are little-endian, so every cell is byte-swapped as it is read.

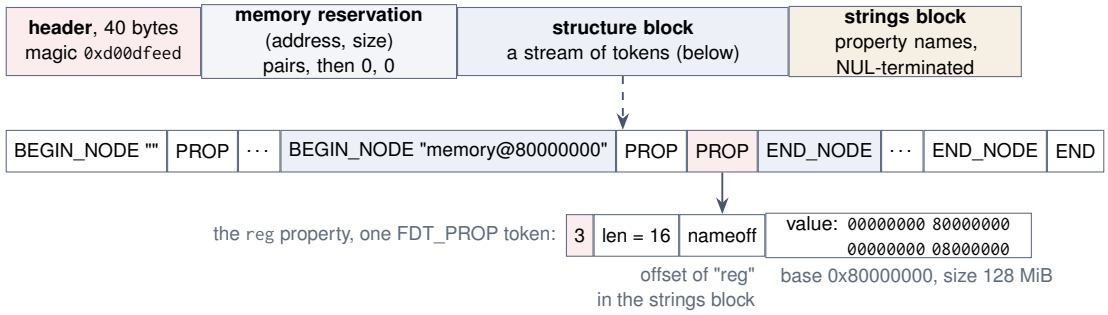


Figure 2: The layout of a flattened devicetree, and the token sequence of the `reg` property of `/memory@80000000`.

2.5 Word size: ILP32 and LP64

i386 and riscv32 use the **ILP32** model: `int`, `long` and pointers are 32 bits. AArch64 uses **LP64**: `int` stays at 32 bits, while `long`, pointers, `size_t` and `uintptr_t` grow to 64. A pointer kept in a `uint32_t` or printed with `%x` worked by accident on x86 and breaks on AArch64. Table 2 lists the differences that bite; the last two even differ between the two 32-bit targets.

Question	i386	riscv32	aarch64
`sizeof(long)`, `sizeof(void *)`, `sizeof(size_t)`	4	4	8
`sizeof(int)`; `sizeof(long long)`	4; 8	4; 8	4; 8
offset of `b` in `struct { uint32_t a; uint64_t b; }`, and the struct's size	4, 12	8, 16	8, 16
is plain `char` signed?	yes	no	no

Table 2: The C data model on the three targets (checked with the compilers' output).

What must *not* change is what leaves the machine. KaganFS has only 8-, 16- and 32-bit fields, which all three little-endian targets lay out the same way, so one disk image works everywhere; a `_Static_assert` on each on-disk structure's size keeps it so. Elsewhere, use `uintptr_t` for an address in an integer, `size_t` for a size, `uint64_t` for physical addresses and lengths from the firmware, and `uint8_t` for raw bytes. The best auditor is the compiler: build the generic code for a target with another data model, and it reports every truncation and format mismatch (Task 16.9).

3 References

Weight	Source	Section	Pages	Relevance
PRIMARY	MOS	§12.3.9 *Useful Techniques*: *Hiding the Hardware*, *Indirection*	1005–1008	Conditional compilation, orthogonal macros, indirection.

Weight	Source	Section	Pages	Relevance
PRIMARY	SPEC	Devicetree Specification (release v0.4), ch. 2 *The Devicetree*: §2.2 structure and conventions, §2.3 standard properties, §2.4 interrupts	–	Nodes, `compatible`, `reg`, cells, `phandle`.
PRIMARY	SPEC	Devicetree Specification, ch. 5 *Flattened Devicetree (DTB) Format*	–	The exact blob layout for the parser of Task 16.10.
SECONDARY	SPEC	Devicetree Specification, ch. 3 *Device Node Requirements*: §3.4 `/memory`, §3.5 `/reserved-memory`, §3.6 `/chosen`	–	What replaces the Multiboot information.
SECONDARY	MOS	§12.3.1 *System Structure* (layered systems, Fig. 12-2)	993–997	Where the hardware-dependent layers belong.
SECONDARY	OSDI	§2.6.11 *Hardware-Dependent Kernel Support*	185–190	How MINIX 3 kept x86-specific C code in a few files.
SECONDARY	OSC	§9.6 *Example: Intel 32- and 64-bit Architectures*; §9.7 *Example: ARMv8 Architecture*	379–384	A preview of the MMUs of Stage V.
OPTIONAL	XV6	Ch. 1 *Operating system interfaces*; ch. 2 *Operating system organization*	–	Introduces xv6-riscv, Phase 17's reference kernel.
PRIMARY	LKD	Ch. 19, *Portability* (word size, data types, alignment, byte order)	379–394	The chapter-length version of Task 16.9's checklist.
OPTIONAL	LKP	Ch. 5, *Cross-compiling a kernel module*	204–216	Cross toolchains from the Linux side.

For devicetree practice, the Linux kernel documentation page *Linux and the Devicetree* (https://docs.kernel.org/devicetree/usage-model.html) explains how a kernel uses the tree at boot.

Prior art: MINIX 3 on ARM, and Linux's arch/ tree

The MINIX 3 of OSDI (version 3.1) ran only on 32-bit x86, but §2.6.11 explains that its hardware-dependent C code was kept in a few files (`exception.c`, `i8259.c`, `protect.c`) to make porting easier. MINIX 3.3.0 (2014) was the first release with ARM support, for the BeagleBoard-xM and the BeagleBone (Cortex-A8, 32-bit ARMv7). In its tree, `minix/kernel/arch/earm` stands next to `minix/kernel/arch/i386` with the same file names (`head.S`, `mpx.S`, `klib.S`, `exception.c`, `protect.c`, `memory.c`, `arch_clock.c`, `kernel.lds`); `i8259.c` exists only on the x86 side, the TI board support only in `earm/bsp/ti`. Linux does the same at scale: `arch/x86`, `arch/riscv` and `arch/arm64` each provide headers under `include/asm`, and `include/asm-generic` supplies a default for whatever an architecture doesn't override.

4 Implementation

Task 16.1 Two more cross compilers, two QEMU boards, dtc and a multi-architecture GDB

Follow section 9 of `docs/setup.md`, *Toolchains for the ports*. Section 9.2 installs the two emulators, `dtc` and a GDB that knows all three architectures. Section 9.3 builds `riscv32-elf` and `aarch64-elf` exactly like the `i686-elf` compiler of Phase 0, with the same binutils and GCC versions, the same prefix and one build folder per target. Only GCC for RISC-V gets two more `configure` options, `--with-arch=rv32imac_zicsr_zifencei` and `--with-abi=ilp32`, which make RV32IMAC and the `ilp32`

ABI its defaults. Run the checks of sections 9.5 and 9.6, then keep both boards' devicetrees for Task 16.10:

terminal

```
mkdir -p tests/data
qemu-system-riscv32 -machine virt,dumpdtb=tests/data/virt-riscv32.dtb
qemu-system-aarch64 -machine virt,dumpdtb=tests/data/virt-aarch64.dtb -cpu cortex-a72
dtc -I dtb -O dts tests/data/virt-riscv32.dtb | less
```

Verify: `riscv32-elf-gcc -v` shows `--with-arch=rv32imac_zicsr_zifencei --with-abi=ilp32` in its configure line; `readelf -h` on a compiled object reports `ELF32` and `RISC-V` for one compiler, `ELF64` and `AArch64` for the other. `qemu-system-riscv32 -machine virt -nographic -bios default` prints the OpenSBI banner with `Domain0 Next Mode : S-mode`, and `dtc` decodes both blobs.

Task 16.2 Find every place x86 leaked

This is the heart of the phase. Before you move a single file, write down every place where code outside `kernel/arch/i386` knows it runs on x86. Search for the usual suspects, then read every hit.

terminal

```
grep -rnE --include='*.[ch]' -e 'struct regs|->(eax|ebx|ecx|edx|esi|edi|ebp|eip|cs|useresp)\b' \
  -e '\bcr[234]\b|invlpg|\b(inb|outb|inl|outl|insw|outsw)\(|\b(cli|sti|hlt)\b|pushf|0x200\b' \
  -e '0xC0000000|KERNEL_VMA|multiboot|rdtsc|pde_t|PG_[A-Z]|0xB8000|vga_|pic_|pit_|0xF4\b' \
  kernel lib include tests | grep -v '^kernel/arch/i386/'
grep -rnE --include='*.[ch]' '\(uint32_t\) *[&a-z_]|uint32_t +[a-z_]*(addr|ptr|len|size)\b' \
  kernel lib include | grep -v '^kernel/arch/i386/'
```

Record each leak in `docs/porting.md`, a working document like the engineering journal, with its kind: **arch** (a register, an instruction, the page-table format), **board** (a device, a firmware structure, a fixed address) or **word** size (a 32-bit type holding an address or a length), and the task that fixes it.

docs/porting.md (the first rows of a typical audit)

```
| file:line                  | leak (kind)                          | fix (task)                | done |
|----------------------------|--------------------------------------|---------------------------|------|
| kernel/syscall/syscall.c:31 | arch: reads r->eax, r->ebx, ...      | tf_sysno, tf_arg (16.5)   | [ ]  |
| kernel/mm/pmm.c:40          | board: walks the Multiboot map       | struct bootinfo (16.6)    | [ ]  |
| kernel/syscall/uaccess.c:6  | word: uint32_t end = start + n       | uintptr_t (16.9)          | [ ]  |
```

Some assumptions have no keyword for grep to find: the stack grows down, pages are 4 KiB, the CPU is little-endian. List them in a second table, *assumptions kept*, and check that all three targets share each one (these three do).

Verify: Every hit of the two searches is in `porting.md`, with a kind and a fix, or marked there as a false positive.

Task 16.3 Move x86 into its corner

Move everything that is x86 or PC into `kernel/arch/i386` with `git mv`, so the history follows each file: the boot code and linker script (Phase 1's `boot/loader.s` and `link.ld` become `boot.s` and `kernel.ld`), descriptor tables, interrupt stubs, PIC, PIT, TSS, paging, the context switch, and the drivers that only exist on a PC (VGA text, PS/2 keyboard, ATA through ports). The x86 headers move from `include/arch/` to `kernel/arch/i386/include/arch/`, and `include/` keeps only generic headers.

Generic code needs no change for this, thanks to the include path. Generic files keep writing `#include <arch/irq.h>`, and `CFLAGS += -Iinclude -Ikernel/arch/$(ARCH)/include` makes the compiler find the header of whichever architecture is being built.

Verify: `include/arch/` no longer exists, `make clean && make test` passes, and `grep -rn 'arch/i386' kernel lib include --include='*.c'` finds nothing outside `kernel/arch/`.

Task 16.4 The architecture interface

Write `include/arch.h`, the contract between the generic kernel and every port; Phases 17 and 18 implement exactly these names. Most functions already exist on x86 under other names: rename them, and let generic code call only these.

include/arch.h

```
#pragma once
#include <stdint.h>
#include <stdbool.h>
#include <bootinfo.h>
#include <arch/trapframe.h>    /* struct trapframe and the inline tf_ accessors */
#include <arch/mmu.h>          /* pte_t, paddr_t and struct arch_space          */

struct task;
struct context;                /* callee-saved registers, on the kernel stack */
struct addrspace;              /* include/vm.h                                */
struct fault_info { uintptr_t addr; bool write, user, present, exec; };

/* boot, interrupts, time */
void     arch_early_init(struct bootinfo *bi);   /* console up, bootinfo filled */
void     arch_init(void);                        /* traps, interrupt controller  */
void     arch_irq_unmask(unsigned irq);
void     arch_irq_mask(unsigned irq);
void     arch_timer_init(unsigned hz);
uint64_t arch_cycles(void);                      /* TSC, time CSR, CNTVCT_EL0    */
uint64_t arch_entropy(void);                     /* seed for the stack canary    */
void     arch_fault_info(const struct trapframe *tf, struct fault_info *fi);

/* tasks, user mode, signals */
void     arch_task_init(struct task *t, void (*entry)(void *), void *arg);
void     arch_switch(struct context **save, struct context *load);
void     arch_user_entry(struct trapframe *tf, uintptr_t entry, uintptr_t sp);
_Noreturn void arch_return_to_user(struct trapframe *tf);
int      arch_signal_frame(struct trapframe *tf, int sig, uintptr_t handler, uintptr_t restorer);
int      arch_sigreturn(struct trapframe *tf);

/* address spaces; flags are VM_READ, VM_WRITE, VM_EXEC, VM_USER, VM_COW */
int      arch_mmu_init_space(struct addrspace *as);
void     arch_mmu_free_space(struct addrspace *as);
int      arch_mmu_map(struct addrspace *as, uintptr_t va, paddr_t pa, unsigned flags);
void     arch_mmu_unmap(struct addrspace *as, uintptr_t va);
bool     arch_mmu_lookup(struct addrspace *as, uintptr_t va, paddr_t *pa, unsigned *flags);
int      arch_mmu_protect(struct addrspace *as, uintptr_t va, unsigned flags);
void     arch_mmu_switch(struct addrspace *as);
void     arch_tlb_flush(uintptr_t va);

/* leaving */
_Noreturn void arch_test_exit(int failed);
_Noreturn void arch_halt(void);
```

The inline headers complete the interface. `<arch/irq.h>` provides `unsigned long irq_save(void)`, `void irq_restore(unsigned long flags)`, `irq_enable()`, `irq_disable()` and `cpu_idle()` (`sti; hlt` on x86); the flags type changes from Phase 9's `uint32_t` to `unsigned long`, the register size everywhere. `<arch/atomic.h>` provides `atomic_xchg()` and `ARCH_HAS_ATOMICS` (1 on x86). Phase 8's `switch_context(&prev->esp, next->esp)` becomes `arch_switch(&prev->ctx, next->ctx)` with a `struct context *ctx`. The x86 assembly doesn't change: the saved ESP already points at the pushed EDI, ESI, EBX, EBP and return address, which is all `struct context` is. `arch_task_init` builds a new task's

first context on its kernel stack: at the top, or just below `t->tf` when generic code has already put a trap frame there, as `fork` does in Task 16.5. Keep the header small: a function joins it only when generic code really needs it.

Verify: `grep -rn 'switch_context\|qemu_exit\|enter_user_mode' kernel --include='*.c' | grep -v '^kernel/arch/'` finds nothing, and `make test` passes.

Task 16.5 One trap frame for generic code

Rename Phase 3's `struct regs` to `struct trapframe`, with the same layout (`isr.s` builds it), and add the accessors. From now on only code in `kernel/arch/i386` may name a field.

kernel/arch/i386/include/arch/trapframe.h

```
struct trapframe {                      /* was struct regs (Phase 3): same layout */
    uint32_t ds;
    uint32_t edi, esi, ebp, esp_at_pusha, ebx, edx, ecx, eax;
    uint32_t int_no, err_code;
    uint32_t eip, cs, eflags;
    uint32_t useresp, ss;               /* valid only for a trap from ring 3 */
};
_Static_assert(sizeof(struct trapframe) == 64, "isr.s depends on this layout");

static inline unsigned long tf_sysno(const struct trapframe *tf) { return tf->eax; }
static inline unsigned long tf_arg(const struct trapframe *tf, int n)
{
    const uint32_t r[6] = { tf->ebx, tf->ecx, tf->edx, tf->esi, tf->edi, tf->ebp };
    return r[n];                        /* the Linux i386 order */
}
static inline void tf_set_ret(struct trapframe *tf, long v) { tf->eax = (uint32_t)v; }
static inline bool tf_from_user(const struct trapframe *tf) { return (tf->cs & 3) == 3; }
/* tf_pc, tf_set_pc and tf_sp in the same style: eip, eip and useresp */
```

Then rewrite the generic users. `syscall_dispatch(tf)`, which each port's trap code calls, sets `current->tf`, checks `tf_sysno(tf)` against the table, calls the entry with six `unsigned long` arguments, `tf_arg(tf, 0)` to `tf_arg(tf, 5)` (enough for every call on every port), and stores the result with `tf_set_ret`. `fork` no longer needs Phase 11's `isr_return` label. It copies the parent's frame to the top of the child's kernel stack, sets `child->tf` to it, calls `tf_set_ret(tf, 0)`, and then `arch_task_init(child, fork_return, tf)`, where `fork_return(void *tf)` only calls `arch_return_to_user(tf)`. The child starts as an ordinary kernel task whose first function returns to user mode.

The page-fault handler gets the same treatment: `arch_fault_info` fills a `struct fault_info` from CR2 and the error code, and generic code decides what to do (Task 16.7). `exec` copies the error code's instruction-fetch bit, which x86 sets only with NX or with SMEP enabled (Task 15.3); on a CPU without SMEP it stays false.

Verify: `grep -rnE -- '->(eax|ebx|eip|cs|useresp)' kernel lib --include='*.c' | grep -v '^kernel/arch/'` finds nothing. The `fork_values` test and all the evil programs of Task 10.9 behave as before.

Task 16.6 Boot information without Multiboot

Generic code must stop reading the Multiboot structure. Define what the kernel learns at boot, whatever the firmware, and let each architecture fill it in. Addresses and lengths are `uint64_t` even on 32-bit targets: physical memory can be larger than the address space, and the RISC-V devicetree uses 64-bit values too.

include/bootinfo.h

```
#define BOOTINFO_MAX_MEM  32
#define BOOTINFO_MAX_MODS 16

enum mem_type { MEM_USABLE = 1, MEM_RESERVED = 2 };

struct mem_region  { uint64_t base, length; enum mem_type type; };
struct boot_module { uint64_t start, end; char name[32]; };  /* physical; end is exclusive */

struct bootinfo {
    struct mem_region  mem[BOOTINFO_MAX_MEM];   unsigned nmem;
    struct boot_module mods[BOOTINFO_MAX_MODS]; unsigned nmods;   /* initrd and other modules */
    uint64_t           kernel_start, kernel_end;    /* physical range of the kernel image   */
    char               cmdline[256];
    const void        *fdt;                         /* the devicetree on the ports, or NULL */
    const char        *console;                     /* a hint: "com1", or /chosen stdout-path */
};
```

On x86, `arch_early_init` calls `bootinfo_from_multiboot(bi, magic, mbi)` in `kernel/arch/i386/multiboot.c`, which does what Phases 2, 5 and 10 did inline: it checks the magic `0x2BADB002`, copies the memory map (flags bit 6) with type 1 as `MEM_USABLE` and everything else as `MEM_RESERVED`, then the modules (bit 3) and the command line (bit 2), all read through `P2V`. It takes the kernel's range from the linker symbols through `V2P`, sets `console` to `"com1"` and leaves `fdt` `NULL`. The PMM becomes `pmm_init(const struct bootinfo *bi)`, and `kmain(struct bootinfo *bi)` never sees Multiboot again.

Verify: The `mem` command prints the same regions and free-frame count as in version 1.0, with `-m 128` and with `-m 512`. `grep -rln multiboot kernel lib include` lists only files under `kernel/arch/i386/`.

Task 16.7 Memory layout and the MMU behind an interface

Move the layout constants into `<arch/memlayout.h>`: `KERNEL_BASE` (`0xC0000000`, Phase 6's `KERNEL_VMA` renamed), `USER_BASE` (`0x00400000`), `USER_STACK_TOP` (`KERNEL_BASE` on x86), `PAGE_SIZE` (moved from `pmm.h`), `DIRECT_MAP_SIZE`, and `P2V` and `V2P` unchanged.

Then split the VMM. Generic code keeps the policy: `struct addrspace` (`include/vm.h`, holding the `struct arch_space` of `<arch/mmu.h>`, on x86 the page directory), the ELF loader, `brk`, stack growth and copy-on-write. Each architecture provides the mechanism, the `arch_mmu_*` functions, and translates the generic flags into its PTE bits. On x86, `VM_WRITE` and `VM_USER` become `PG_WRITE` and `PG_USER`, `VM_COW` becomes Phase 11's `0x200` (a bit left to the OS), and `VM_READ` and `VM_EXEC` need no bit, because 32-bit paging has neither a read bit nor an NX bit. Every `arch_mmu_` call leaves the TLB correct for the address it changed; `arch_mmu_init_space` copies the kernel half (PDEs 768–1023), and `arch_mmu_switch` replaces the CR3 load in `schedule()`. The copy-on-write case of Phase 11 now reads as generic code, and Phases 17 and 18 inherit it unchanged:

kernel/mm/fault.c (copy-on-write)

```
static bool cow_fault(struct addrspace *as, uintptr_t va)    /* va: page-aligned */
{
    paddr_t old, copy;
    unsigned fl;
    if (!arch_mmu_lookup(as, va, &old, &fl) || !(fl & VM_COW))
        return false;
    fl = (fl & ~VM_COW) | VM_WRITE;
    if (pmm_refcount(old) == 1)                      /* last user: just take it back */
        return arch_mmu_protect(as, va, fl) == 0;
    if (!(copy = pmm_alloc_frame()))
        return false;
    memcpy(P2V(copy), P2V(old), PAGE_SIZE);
    arch_mmu_unmap(as, va);
    arch_mmu_map(as, va, copy, fl);
```

```
    pmm_unref(old);
    return true;
}
```

`page_fault(tf)`, called by each port's trap code, gets a `struct fault_info` from `arch_fault_info`, tries `cow_fault` for a write to a present page below `KERNEL_BASE`, then Phase 11's lazy heap and stack growth for a page that isn't present. Only then does it print the report (address, access, mode, `tf_pc(tf)`) and send `SIGSEGV`, or panic for a fault in the kernel.

The uaccess contract of Phases 10 and 11 belongs to the same generic code, so that every port inherits it instead of rewriting it. `kernel/syscall/uaccess.c` keeps `copy_from_user`, `copy_to_user`, `strncpy_from_user` and `fault_in_user_page`; they look pages up with `arch_mmu_lookup` and generic `VM_` flags, resolve lazy and copy-on-write pages through the same functions as `page_fault`, and still reject a write to a page without `VM_WRITE` or `VM_COW` with `-EFAULT`. The only machine-specific part is a pair of hooks around the copy itself, `arch_uaccess_begin()` and `arch_uaccess_end()` in `<arch/uaccess.h>`: empty on x86, or `stac` and `clac` when SMAP is enabled (Task 15.3). Phases 17 and 18 fill them in for their own user-access switches. The evil programs go the same way: those that only pass bad arguments to system calls (`badptr`, `hugelen`, `ro_read`, `lazy_read`, `cow_read` and the rest) become plain C in `user/test/`, and the few that need a machine instruction or a kernel address (`cli`, `kpeek`, `badsys`) get a short body per architecture in `user/arch/$(ARCH)/`.

Verify: `grep -rnE 'pde_t|PG_[A-Z]|invlpg|cr3' kernel lib include | grep -v '^kernel/arch/i386/'` finds nothing. The Phase 6 VMM tests, `cow_frames` and the `brk` test pass, and `crash null` still prints a decoded report. `make test ARCH=i386` still passes with uaccess and the evil programs in their generic places, including `ro_read`, `lazy_read` and `cow_read`.

Task 16.8 Consoles and input that any board can provide

Phase 2's console layer called the VGA and serial drivers by name. Make it a registry, and give input one entry point: the PS/2 keyboard's handler (x86 only) and a UART's receive interrupt (every board) both call `tty_input_char`, which feeds Phase 14's line discipline. Then split the serial driver, as the portability note of Task 2.6 suggested. Everything about the 16550 (init, transmit, receive) moves to `kernel/drivers/uart16550.c` and reaches the registers only through two accessors. The RISC-V `virt` board has the same chip, memory-mapped, so Phase 17 reuses the file with a base address from the devicetree.

include/console.h and kernel/drivers/uart16550.c (core)

```
struct console {
    const char     *name;                       /* "vga", "com1", "uart0"          */
    void          (*putc)(struct console *c, char ch);
    struct console *next;
};
void console_register(struct console *c);       /* kprintf now also prints here    */
void tty_input_char(char c);                    /* every input device ends up here */

enum { UART_DATA = 0, UART_IER = 1, UART_FCR = 2, UART_LCR = 3, UART_MCR = 4, UART_LSR = 5 };

struct uart16550 {
    struct console con;
    uintptr_t      base;              /* an I/O port, or the registers' virtual address */
    unsigned       shift;             /* register n is at base + (n << shift)           */
    uint8_t      (*read)(const struct uart16550 *u, unsigned reg);
    void         (*write)(const struct uart16550 *u, unsigned reg, uint8_t val);
};

uint8_t uart_mmio_read(const struct uart16550 *u, unsigned reg)
{
    return *(volatile uint8_t *)(u->base + (reg << u->shift));
```

```
}

void uart16550_irq(struct uart16550 *u)          /* the port's IRQ handler calls this  */
{
    while (u->read(u, UART_LSR) & 0x01)          /* data ready: drain the receive FIFO */
        tty_input_char((char)u->read(u, UART_DATA));
}
```

`uart_putc`, the `con.putc` of every 16550, waits for bit 5 of the line status register and writes the data register, both through the accessors. The x86 side supplies port I/O accessors (`inb` and `outb` on `base + reg`) in `kernel/arch/i386/uart_pio.c` and registers COM1 at `0x3F8`. Turn on serial *input* on x86 too, because it is the path the ports depend on: set bit 0 of the interrupt enable register (received data available), write `0x0B` instead of Phase 2's `0x03` to the modem control register (OUT2 connects the UART's interrupt line to the PIC on a PC), unmask IRQ 4, and call `uart16550_irq` from its handler.

Verify: Boot output appears on the screen and in the `-serial stdio` terminal exactly as before. Keys typed into the terminal reach `kash` through IRQ 4, with editing and Ctrl+C working as on the PS/2 keyboard.

Task 16.9 Make the generic code 64-bit clean

The i386 kernel has been built with `-Wall -Wextra -Wformat=2 -Werror` and a format-checked `kprintf` (`__attribute__((format(printf, 1, 2)))`) since Phase 2; now the same checks run with all three compilers. The fixed-width types differ between them: `uint32_t` is `unsigned long` on `i686-elf` and `riscv32-elf` but `unsigned int` on `aarch64-elf`, so no single conversion fits it everywhere. In generic code, cast a fixed-width value to `unsigned long` and print it with `%lx` or `%lu`, and print a `uint64_t` as `unsigned long long` with `%llx`. GCC warns about casts between pointers and integers of a different size by default (`-Wpointer-to-int-cast`, `-Wint-to-pointer-cast`). Teach `kvprintf` the `l`, `ll` and `z` modifiers (`%lx`, `%llx`, `%zu`). Fix what the warnings show with the rules of the concepts section, and watch plain `char`: `if (c < 0)` is always false on both ports (`-Wextra` says so, through `-Wtype-limits`), and a byte above `0x7F` used as an index goes negative on x86.

On x86 alone, half of these mistakes stay silent, so add a target that compiles every generic file with both new compilers and `-Werror`. Nothing is linked; the point is to let a 64-bit compiler read every line.

Makefile (check-generic)

```
CHECK_CFLAGS := -std=gnu11 -ffreestanding -O2 -Wall -Wextra -Werror -Wformat=2 \
                -Iinclude -Ikernel/arch/none/include
GENERIC_SRCS := $(filter-out kernel/arch/%,$(wildcard kernel/*.c kernel/*/*.c lib/*.c))

build/check/riscv32/%.o: %.c
    @mkdir -p $(dir $@)
    riscv32-elf-gcc -march=rv32imac_zicsr_zifencei -mabi=ilp32 $(CHECK_CFLAGS) -c $< -o $@
build/check/aarch64/%.o: %.c
    @mkdir -p $(dir $@)
    aarch64-elf-gcc -mgeneral-regs-only $(CHECK_CFLAGS) -c $< -o $@

check-generic: $(foreach t,riscv32 aarch64,$(patsubst %.c,build/check/$(t)/%.o,$(GENERIC_SRCS)))
    @echo "generic code compiles for riscv32 and aarch64"
.PHONY: check-generic
```

Which arch/ headers does check-generic use?

The real RISC-V and ARM headers don't exist until Phases 17 and 18, so create `kernel/arch/none/include/arch/`: compile-only stubs with plausible types (a trap frame of `unsigned long` registers, a 64-bit high-half `KERNEL_BASE` when pointers are 64 bits, accessors that

compile but mean nothing). Only `check-generic` uses them, and nothing links against them. Writing the real port headers now would mean designing the RISC-V trap frame before reading the privileged specification. Phase 17 switches the `riscv32` rule to its real headers; Phase 18 does the same for `aarch64` and deletes `arch/none`.

Verify: `make check-generic` ends with its message and no warnings. Put `uint32_t x = (uint32_t)ptr;` into any generic file: the aarch64 rule now fails with GCC's `cast from pointer to integer of different size`. Take it out again, and `make ARCH=i386 test` still passes.

Task 16.10 Read a devicetree

Write `lib/fdt.c`, a small read-only parser that both ports call from `arch_early_init` to fill `struct bootinfo` and find their devices. Keep it free of kernel dependencies (only `<stdint.h>`, `<stddef.h>` and string functions that both `lib/string.c` and the host C library have), so it can be tested on Linux as Phase 0 tested the warm-up library.

Its interface in `include/fdt.h` is `fdt_check(fdt)`, `fdt_next_node(fdt, node, &depth)`, `fdt_find_node(fdt, path)`, `fdt_getprop(fdt, node, name, &len)`, `fdt_next_compatible(fdt, after, compat)`, `fdt_reg(fdt, node, i, &base, &size)` and `fdt_be32(p)`; nodes are `int` offsets and -1 means "none". `fdt_check` compares the magic number, refuses a `last_comp_version` above 17 (the version this parser knows), and checks that every block lies inside `totalsize`. A node is the offset of its `FDT_BEGIN_NODE` token in the structure block, and everything else is built on one walker that steps from node to node, keeps track of the depth, and never trusts a length:

lib/fdt.c (the core)

```
#define FDT_MAGIC 0xd00dfeedu
enum { FDT_BEGIN_NODE = 1, FDT_END_NODE = 2, FDT_PROP = 3, FDT_NOP = 4, FDT_END = 9 };

uint32_t fdt_be32(const void *p)                /* byte by byte: any alignment, any CPU */
{
    const uint8_t *b = p;
    return (uint32_t)b[0] << 24 | (uint32_t)b[1] << 16 | (uint32_t)b[2] << 8 | b[3];
}
static uint32_t hdr(const void *fdt, int i) { return fdt_be32((const uint8_t *)fdt + 4 * i); }
static uint32_t align4(uint32_t n) { return (n + 3) & ~3u; }

int fdt_next_node(const void *fdt, int node, int *depth)
{
    const uint8_t *s = (const uint8_t *)fdt + hdr(fdt, 2);   /* off_dt_struct  */
    uint32_t end = hdr(fdt, 9), off, len;                    /* size_dt_struct */
    if (node < 0) {                                          /* the root node  */
        *depth = 0;
        return fdt_be32(s) == FDT_BEGIN_NODE ? 0 : -1;
    }
    off = node + 4 + align4(strlen((const char *)s + node + 4) + 1);   /* token, name */
    while (off + 4 <= end) {
        switch (fdt_be32(s + off)) {
        case FDT_BEGIN_NODE: (*depth)++; return (int)off;
        case FDT_END_NODE:   (*depth)--; off += 4; break;
        case FDT_NOP:        off += 4; break;
        case FDT_PROP:                                       /* len, nameoff, value */
            if (end - off < 12 || (len = fdt_be32(s + off + 4)) > end - off - 12)
                return -1;
            off += 12 + align4(len);
            break;
        default:             return -1;                      /* FDT_END, or corrupt */
        }
    }
    return -1;
```

```
}
```

`fdt_getprop` walks the properties after a node's name up to its first child. `fdt_find_node` matches a path one component per depth. `fdt_reg` reads the *parent's* `#address-cells` and `#size-cells` (the defaults are 2 and 1) and joins one or two cells into each 64-bit value. Test it in `tests/host/test_fdt.c`, compiled with the host `gcc` and `-fsanitize=address,undefined` as in Phase 0, against facts from the `dtc` output of the riscv32 blob: `/memory@80000000` has base `0x80000000` and size 128 MiB (QEMU's default `-m`); the first `ns16550a` node has base `0x10000000` and `interrupts` 10; `/cpus` has a `timebase-frequency` of 10 000 000; there are eight `virtio,mmio` nodes; and `/chosen` has a `stdout-path`. Write the same kind of test for the aarch64 blob (its memory node, the PL011, the timer). Finally, try to break the parser: flip a byte of the magic, make `off_dt_struct` plus `size_dt_struct` point past `totalsize`, set a property's length to `0xFFFFFFF0`. Each must give an error, never a crash or an endless loop.

The blob is a new input from firmware, and the ports bring more. Add them to `docs/security/threat-model.md` before you write the code that trusts them. The devicetree crosses the firmware-to-kernel boundary, so it is parsed like a disk image, never trusted. OpenSBI runs in M-mode below the RISC-V kernel and sees all of memory, so it is part of the TCB, like the BIOS and GRUB on x86, and its source and version belong in the toolchain record. Virtio devices read and write guest-physical memory by DMA, wherever their descriptors point, and nothing stops a malicious device from writing anywhere. QEMU's devices are trusted, and DMA isolation with an IOMMU stays out of scope; write both down.

Verify: The host tests pass on both blobs under the address and undefined-behaviour sanitizers, the corrupted blobs are rejected, and `lib/fdt.c` also compiles in `make check-generic`.

Task 16.11 One Makefile, three architectures

Make the architecture a build variable. What differs per architecture moves into `kernel/arch/$(ARCH)/arch.mk`: cross prefix, extra flags, sources, linker script, debugger, how to run QEMU, and how a test run reports success. The `GDB` line from `docs/setup.md`, section 4, moves there too (section 9.4). Output goes to `build/$(ARCH)/`, so objects of different architectures never mix.

Makefile (the parts that change), then kernel/arch/i386/arch.mk

```
ARCH     ?= i386
ARCH_DIR := kernel/arch/$(ARCH)
BUILD    := build/$(ARCH)
ifeq ($(wildcard $(ARCH_DIR)/arch.mk),)
$(error unknown ARCH '$(ARCH)': there is no $(ARCH_DIR)/arch.mk)
endif
include $(ARCH_DIR)/arch.mk
CC     := $(CROSS)gcc
CFLAGS += $(ARCH_CFLAGS) -Iinclude -I$(ARCH_DIR)/include
OBJS   := $(addprefix $(BUILD)/,$(addsuffix .o,$(basename $(GENERIC_SRCS) $(ARCH_SRCS))))
# run, debug and test use $(QEMU) $(QEMU_FLAGS); test adds $(QEMU_TEST_FLAGS)
# and compares QEMU's exit status with $(TEST_PASS_STATUS)

# ---- kernel/arch/i386/arch.mk ----
CROSS            := i686-elf-
ARCH_SRCS        := $(wildcard kernel/arch/i386/*.c kernel/arch/i386/*.s)
LDSCRIPT         := kernel/arch/i386/kernel.ld
GDB              := $(call find_tool,i386-elf-gdb gdb)
QEMU             := qemu-system-i386
QEMU_FLAGS       := -kernel $(BUILD)/kernel.elf -serial stdio
QEMU_TEST_FLAGS  := -append "test" -display none -no-reboot \
                    -device isa-debug-exit,iobase=0xf4,iosize=0x04
# isa-debug-exit: QEMU exits with 2 * value + 1, and a passing run writes 0
TEST_PASS_STATUS := 1
# ... plus the NASM pattern rule for the .s files, which only x86 has
```

Do the same for user space: `crt0.s` and the system call stubs move to `user/arch/i386/`, and user programs get their own flags without `-Ikernel`. Phases 17 and 18 add the `arch.mk` files for `riscv32-elf-` and `aarch64-elf-`; until then `make ARCH=riscv32` stops with the error.

Teach CI the same variable. The job of Phase 7 becomes a matrix over `ARCH`, with one entry that runs `make test` with that `ARCH`, the evil programs and a short fuzzer run for each architecture that has an `arch.mk`, and one that runs `make check-generic` and the host tests. Today the matrix has only `i386`; Phases 17 and 18 add `riscv32` and `aarch64`, and their cross compilers join the cached toolchain.

Verify: `make` and `make ARCH=i386` build the same `build/i386/kernel.elf`; `make ARCH=i386 run`, `debug` and `test` work; `make ARCH=sparc` prints the error; nothing is written to `build/` outside `build/i386/` and `build/check/`. The CI matrix is green for `i386` and for `check-generic`.

Task 16.12 Prove that nothing broke

A refactoring is finished when the behaviour is provably the same. Run Phase 15's safety net against the new tree and compare with the v1.0 benchmark table.

1. `make clean && make ARCH=i386 test`: every ktest and user-space test passes.
2. Four fuzzers and the stress script of Task 15.1 for ten minutes: no panic, no growth in memory use.
3. The benchmarks of Task 15.5, above all the null system call (now through `syscall_dispatch` and six `tf_arg` calls) and the context switch (now `arch_switch`).
4. `make check-generic` and the FDT host tests, from now on before every commit.

If the null system call got slower, read the machine code before guessing: `i686-elf-objdump -d build/i386/kernel/syscall/syscall.o` must show no `call` to a `tf_` function (an accessor declared without `static inline` ends up out of line). Record the numbers in `docs/porting.md`.

Verify: All four items pass, and every benchmark is within a few per cent of its Phase 15 value (repeat the runs and compare medians before deciding that a difference is real).

5 Testing and debugging

Symptom	Likely cause	Resolution
A change to an x86 header has no effect, or two definitions clash	A stale `include/arch/` directory, or a leftover `-Iinclude/arch`, is found before `kernel/arch/$(ARCH)/include`	Delete the old directory; print the search path with `i686-elf-gcc -E -v` and check its order.
The child of `fork` crashes or returns garbage	`arch_task_init` built the first context over the trap frame at the top of the kernel stack, or code still writes `tf->eax`	Build the context below `t->tf` when it is set; grep for field names outside `kernel/arch`.
`fdt_check` fails; the magic reads `0xedfe0dd0`	The header was read as a little-endian `uint32_t`	Read every cell through `fdt_be32`.
RAM above 4 GiB, or a large region, is missing or wrong on one target	A `uint64_t` base or length passed through a `uint32_t` or `uintptr_t` on the way into `bootinfo`	Keep firmware values 64-bit until the PMM clips them to what it can map.
Benchmarks several per cent slower	Accessors became function calls, or `arch_switch` gained work	Look for `call` in `objdump -d`; keep hot helpers `static inline`.

6 Exit criteria

Exit criteria: Portable core

- ☐ Generic code is free of x86: every row of the audit in `docs/porting.md` is done, and the searches of Task 16.2 find nothing outside `kernel/arch/i386`.
- ☐ `make ARCH=i386 test` passes; ten minutes of fuzzing cause no panic; the benchmarks are within a few per cent of Phase 15.
- ☐ uaccess and the evil programs live in generic code behind `arch_uaccess_begin/end`; CI runs a matrix over `ARCH`; the threat model covers the devicetree, OpenSBI and virtio DMA.
- ☐ `make check-generic` compiles every generic file for `riscv32` and `aarch64` without warnings.
- ☐ The FDT parser passes its host tests on both QEMU blobs under the sanitizers.
- ☐ `include/arch.h`, the trap frame accessors, `struct bootinfo`, `arch_mmu_*`, `console_register` and `tty_input_char` are in place; serial input works on x86.
- ☐ Both new cross compilers, both QEMU boards, `dtc` and a GDB for all three architectures are installed and smoke-tested.

Review questions

1. What is the difference between the architecture and the board? Why does x86 keep both in one directory?
2. Why are some parts of the interface inline headers and others functions in `include/arch.h`? What would happen to the null system call if `tf_arg` were a function?
3. Where in Kagan is an `#ifdef` most tempting, and what replaces it?
4. The devicetree is big-endian, and all three targets are little-endian. What would go wrong without `fdt_be32`, and how would the failure show?
5. Why can one KaganFS image work on all three targets, and which kinds of change to its structures would break that?
6. Which subsystems needed no change at all, and what does that say about their design?

7 Extensions

Add a fourth architecture that is not a machine: `kernel/arch/host` runs the generic kernel as a Linux process, with `mmap` standing in for page tables, so the ktests of the heap, the scheduler and KaganFS run in a second under the sanitizers (User-Mode Linux does this for all of Linux). Run `make check-generic` and the host tests from a Git pre-commit hook. Generate the system call table, the user stubs of every architecture and `docs/syscalls.md` from one list. Or add Clang to `check-generic`: `clang --target=riscv32-unknown-elf` and `--target=aarch64-unknown-elf` need no toolchain build, and a second compiler finds different mistakes.

Next: **Phase 17 The RISC-V Port** →

References

LOB Erik Helin, Adam Renberg. *The little book about OS development*, 2015. The hands-on x86 guide. Free at https://littleosbook.github.io.

OSTEP Remzi H. Arpaci-Dusseau, Andrea C. Arpaci-Dusseau. *Operating Systems: Three Easy Pieces*, v0.90, Arpaci-Dusseau Books, 2015. Main theory text. Free at https://pages.cs.wisc.edu/~remzi/OSTEP/.

OSC A. Silberschatz, P. B. Galvin, G. Gagne. *Operating System Concepts*, 10th ed., Wiley, 2018. Reference theory text.

MOS Andrew S. Tanenbaum, Herbert Bos. *Modern Operating Systems*, 4th ed., Pearson, 2015. Hardware, I/O, design, security.

OSDI Andrew S. Tanenbaum, Albert S. Woodhull. *Operating Systems: Design and Implementation*, 3rd ed., Pearson Prentice Hall, 2006. MINIX 3 with full source listing.

LKD Robert Love. *Linux Kernel Development*, 3rd ed., Addison-Wesley, 2010. How Linux implements each subsystem.

LKP Kaiwan N. Billimoria. *Linux Kernel Programming*, Packt, 2021. Modern kernel practice: logging, allocators, the scheduler, synchronisation.

LOB and **OSTEP** are freely available; the roadmap lists ISBNs for the others. Where sources disagree, the design follows the newer one when its method also suits Kagan. Page numbers are the *printed* page numbers of these editions, not PDF page indices. With another edition, or with the free **OSTEP** chapters (each one starts at page 1), find the reading by its chapter and section numbers. MINIX source references like `kernel/proc.c (07400)` give the file and its starting line in **OSDI** Appendix B.

Port references (Stage V)

XV6 Russ Cox, Frans Kaashoek, Robert Morris. *xv6: a simple, Unix-like teaching operating system*, RISC-V edition. The companion book to MIT's xv6-riscv kernel. Free at https://pdos.csail.mit.edu/6.1810/; source at https://github.com/mit-pdos/xv6-riscv.

RISCV *The RISC-V Instruction Set Manual*, Volume I (Unprivileged ISA) and Volume II (Privileged Architecture); *RISC-V Supervisor Binary Interface Specification*. Free at https://riscv.org/specifications/ratified/ and https://github.com/riscv-non-isa/riscv-sbi-doc.

ARM *Arm Architecture Reference Manual for A-profile architecture* (DDI 0487), and Arm's *Learn the architecture* guides: AArch64 exception model, AArch64 memory management, Generic Timer, GIC. Free at https://developer.arm.com/documentation.

SPEC *Devicetree Specification* (https://www.devicetree.org/specifications/); *Virtual I/O Device (VIRTIO)*, OASIS (https://docs.oasis-open.org/virtio/virtio/); the QEMU documentation for the RISC-V and Arm `virt` boards (https://www.qemu.org/docs/master/system/).

All four are free. Specifications are revised often, so the reference tables cite them by chapter and section title rather than by page.